

Code Roulette: How Prompt Variability Affects LLM Code Generation

Andrei Paleyes
Computer Science and Technology
University of Cambridge
Cambridge, United Kingdom
ap2169@cl.cam.ac.uk

Diana Robinson
Computer Science and Technology
University of Cambridge
Cambridge, United Kingdom
dmpr3@cam.ac.uk

Radzim Sendyka
Computer Science and Technology
University of Cambridge
Cambridge, United Kingdom
rs2071@cam.ac.uk

Christian Cabrera
Computer Science and Technology
University of Cambridge
Cambridge, United Kingdom
chc79@cam.ac.uk

Neil D. Lawrence
Computer Science and Technology
University of Cambridge
Cambridge, United Kingdom
ndl21@cam.ac.uk

Abstract

Code generation is one of the most active areas of application of Large Language Models (LLMs). While LLMs lower barriers to writing code and accelerate development process, the overall quality of generated programs depends on the quality of given prompts. Specifically, functionality and quality of generated code can be sensitive to user's background and familiarity with software development. It is therefore important to quantify LLM's sensitivity to variations in the input. To this end we propose an evaluation pipeline for LLM code generation with a focus on measuring sensitivity to prompt augmentations, completely agnostic to a specific programming tasks and LLMs, and thus widely applicable. We provide extensive experimental evidence illustrating utility of our method and share our code for the benefit of the community.

CCS Concepts

• **Software and its engineering** → **Automatic programming**; • **Computing methodologies** → **Artificial intelligence**.

Keywords

large language models, code generation, sensitivity analysis

ACM Reference Format:

Andrei Paleyes, Diana Robinson, Radzim Sendyka, Christian Cabrera, and Neil D. Lawrence. 2026. Code Roulette: How Prompt Variability Affects LLM Code Generation. In *3rd International Workshop on Large Language Models For Code (LLM4Code '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3786181.3788724>

1 Introduction

Since the emergence of Large Language Models (LLMs), one of their most widespread applications is code generation [37]. Ability of LLMs to generate code has the potential to bring many advantages, such as lowering the barriers to coding, democratising and accelerating software development [27]. Notably, requirements for

generated code are formulated by end users in natural language, often lacking any particular structure. In a traditional software development process software engineers collect such requirements from different stakeholders and generate a system specification that satisfies end-user requirements. It is a trust-based relationship between users and engineers.

With the onset of LLMs people with all kinds of backgrounds and coding expertise can use a chat interface to generate code directly. However, at the present moment of LLM development we cannot trust these models as we would trust another human being, because we cannot be certain the model understands our requirements. An important step to build such trust is to make LLMs safer and better understand their behaviour in the context of code generation. In this paper we describe a procedure to aid such understanding, specifically to measure sensitivity of a LLM code generation to variability in the input prompt.

Investigating how sensitive LLM code outputs are to differences in input prompts is important because it will determine what kind of actions are needed to produce the same code outputs from varying prompt inputs that describe the same requirement. For example, we may need to provide more guidance to users, such as follow-up questions as they provide the prompts, as part of the coding pipeline. Or this might involve techniques such as generating other related inputs or other related outputs based on distance and averaging them together to regularise the output code generated. Testing how sensitive LLM outputs are to changes in inputs allows us to understand the scope of this challenge to standardise code outputs to varying descriptions of the same requirement. This paper makes strides towards addressing this challenge. Specifically, we put forth following contributions:

- (1) An evaluation procedure to measure LLM's sensitivity on code generation tasks.
- (2) A sensitivity analysis evaluation of multiple popular LLMs.
- (3) Openly available code that other researches can use to replicate our work and build on it¹.



This work is licensed under a Creative Commons Attribution 4.0 International License. *LLM4Code '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2412-1/26/04

<https://doi.org/10.1145/3786181.3788724>

¹<https://github.com/apaleyes/code-gen-sensitivity>

2 Motivation

Intuitively, our work is motivated by the observation that people can express their thoughts differently depending on their background, education, experience and other contexts. This notion includes the way humans write prompts to interact with LLMs. In this section we present evidence from the literature confirming this observation.

Recent research by Chen et al. [3] investigated robustness of code LLMs to variations that could be expected in real world application based on a literature review and survey of professionals who use LLMs for code. They identified 18 categories of these perturbations such as extra space outside of words. They highlight an example of a prompt for LLM code generation tool, StarCoder, where a tiny change to the prompt with the repetition of the word “it” resulted in incorrect code generated. We anticipate much larger prompt variability from differences in how people decompose problems that they are trying to build code for based on their background knowledge and experience, including what technical concepts about programming they have available to them in their mental models of both programming languages and LLMs.

There is significant evidence that decomposing a problem and thinking about it in a way that can be translated into code is a nontrivial skill. As discussed below, work in computational thinking and research into teaching programming highlights this.

Computational thinking is a type of analytical thinking that draws on concepts fundamental to computing [34]. It involves choosing the right abstraction to describe a problem in a way that can be solved by a computer. This style of thinking is much discussed in educational contexts as a way of helping people to build or understand these solutions and apply them in other context (see for example [22]).

Problem decomposition is an important skill in programming, as a way of formulating subgoals to implement in code [24]. As LLMs become intertwined in the process of generating code, educators are adapting their courses to incorporate them. For example, Vadaparty et al. designed a CS1 course for programming using LLMs which changed the emphasis from syntax toward explaining and testing code generated from LLMs and decomposing complex problems into small pieces that can be used to craft prompts for an LLM [32]. Of their many learning goals, they identified as essential problem decomposition, even if instructors only adopt part of the course.

Programmers rely on mental models of programs and programming concepts to guide their work [11]. As programming shifts to LLMs, mental models will also shift. Some work that illustrates this is by Liang et al. [18], who found that prompt programming differed from traditional software development in that programmers were relying on mental models of LLMs rather than mental models of code. Ehsani et al. explored where developers understanding of LLM responses can come apart and developed a tool to offer templates to developers to improve prompts and enable LLM-driven issue resolution [7].

We can also see this effect of differences in background and knowledge impacting prompt development in multidisciplinary software teams. For example, in a study on prototyping with generative AI tools that discussed differences in approaches to creating prompts between team members such as UX researchers, AI engineers, and product managers [31].

While differences in problem decomposition and mental models between users with varying backgrounds represent an important area for future investigation, this work focuses on a more fundamental question: how sensitive are LLMs to basic variations in prompt text that preserve the same underlying requirement? As a first step toward understanding prompt variability, we examine sensitivity to simple textual perturbations (typos, synonyms, and paraphrasing) that any user might introduce regardless of their background. In the following sections we confirm the sensitivity of LLM generated code to synthetic changes to prompts, with important implications for development of code with LLMs.

3 Methodology

We designed an evaluation pipeline to investigate the sensitivity of LLM generated code to fluctuations in the prompts. In this section we describe this pipeline and motivate key design decisions behind it.

3.1 Overview of the pipeline

Let $\mathcal{P}$ denote the set of all possible prompts representing coding tasks, and let $\mathcal{M} : \mathcal{P} \rightarrow \mathcal{C}$ be a function that maps a prompt to generated code, where $\mathcal{C}$ represents the space of all code snippets. In the context of this work, $\mathcal{M}$ is a LLM.

To systematically evaluate sensitivity, we define an augmentation function $\mathcal{F} : \mathcal{P} \times [0, 1] \rightarrow \mathcal{P}$ that perturbs a rate $r \in [0, 1]$ of the input prompt, thereby producing a modified prompt. Additionally, we employ a distance function $\mathcal{D} : \mathcal{C} \times \mathcal{C} \rightarrow [0, 1]$ that quantifies the dissimilarity between two code snippets, yielding a normalized distance metric. Specific realizations of both $\mathcal{F}$ and $\mathcal{D}$ are discussed in subsequent sections. While our current experiments use lexical augmentations (typos, synonyms, paraphrasing), the pipeline is designed to accommodate more sophisticated augmentation methods that model differences in problem decomposition or conceptual framing.

The evaluation procedure operates as follows. Given a single prompt $p \in \mathcal{P}$, we first establish a reference baseline by generating n independent code samples from the unperturbed prompt. These reference outputs serve as ground truth representations of the model’s behavior on the original task specification. We then systematically vary the augmentation rate parameter from zero to one in discrete increments. The variation in the augmentation rate r generates code outputs that are progressively more perturbed versions of the original prompt. For each augmentation level, we compute the pairwise distances between all generated outputs and the reference baseline set, subsequently aggregating these measurements to obtain an overall distance metric. This metric quantifies how the model’s output distribution changes as a function of prompt augmentation rate, thus providing insight into the model’s sensitivity to input variations. Formal description of the pipeline is given in Algorithm 1.

3.2 Data

We use three datasets for evaluation experiments.

LeetCode (Old) dataset consists of programming tasks from the popular website LeetCode. LeetCode tasks is a popular benchmark for code generation, and is widely used in scientific literature.

Algorithm 1 Evaluation Pipeline for Sensitivity of LLM Code Generation

Require: Prompt p , model $\mathcal{M}$, augmentation function $\mathcal{F}$, distance function $\mathcal{D}$, number of reference samples n , augmentation rate step δ

- 1: Initialize empty set $R \leftarrow \emptyset$
- 2: Generate reference code set: $C_{ref} \leftarrow \{c_i = \mathcal{M}(p) : i = 1, \dots, n\}$
- 3: **for** $r \leftarrow 0$ to 1 step δ **do**
- 4: Generate augmented code set: $C_{aug} \leftarrow \{c_j = \mathcal{M}(\mathcal{F}(p, r)) : j = 1, \dots, n\}$
- 5: Initialize distance sum: $S \leftarrow 0$
- 6: Initialize count: $N \leftarrow 0$
- 7: **for each** $c_{ref} \in C_{ref}$ **do**
- 8: **for each** $c_{aug} \in C_{aug}$ **do**
- 9: Compute pairwise distance: $d \leftarrow \mathcal{D}(c_{ref}, c_{aug})$
- 10: $S \leftarrow S + d$
- 11: $N \leftarrow N + 1$
- 12: **end for**
- 13: **end for**
- 14: Compute average distance: $\bar{d}_r \leftarrow S/N$
- 15: Store result: $R \leftarrow R \cup \{(r, \bar{d}_r)\}$
- 16: **end for**
- 17: **return** R

We have compiled a set of 20 tasks randomly sampled from this benchmark². Unfortunately the most commonly circulated LeetCode tasks are included in the training data of most modern LLMs, which means they achieve almost 100% correct performance on these problems. This issue is known in the literature as “data contamination”, and as shown by Sendyka et al. [29] does not improve even with complete obfuscation of text.

LeetCode (New) dataset is the list of 20 programming tasks posted on LeetCode in March 2025, that we have collected manually. Matching their publication dates and official training cut-off dates of the LLMs we used, we have high confidence these tasks were not used in the training data for LLM models we evaluated.

Our Dataset is the collection of 22 programming tasks we have created ourselves, spanning topics of simulations, algorithms, data science, application development, and games. We made a deliberate effort to specify details that would separate our tasks from common programming problems found online. Problems in this dataset do not have strictly one correct answer, and are designed to be open ended, which is significantly different from LeetCode exercises. The dataset is fully available in the repository accompanying this paper³.

3.3 Prompt Augmentation methods

We apply three augmentation methods to the prompts: keyboard typos, synonyms, and paraphrasing.

Keyboard typos augmentation randomly replaces *rate* characters in the prompt with an adjacent key on the QWERTY keyboard.

²The particular instance of the dataset we used is published on HuggingFace at <https://huggingface.co/datasets/NyanDoggo/leetcode>

³Full dataset is available at <https://github.com/apaleyes/code-gen-sensitivity/blob/main/Sensitivity%20of%20LLMs%20tasks%20dataset.md>

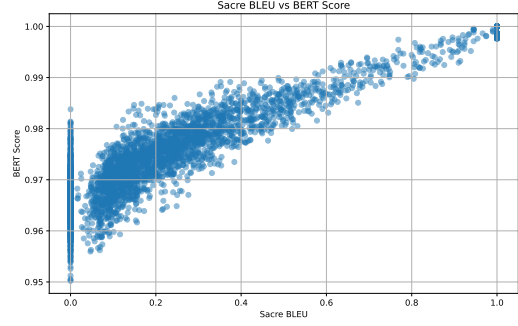


Figure 1: Diversity (i.e., Sacre BLEU) versus semantic similarity (i.e., BERT Score) of the generated paraphrases against the original prompt for Our Dataset. The generated paraphrases are semantically similar, while textually diverse.

This method attempts to simulate typos that a human can make while typing text on a keyboard. We use the NLPaug library to implement this method [19].

Synonyms augmentation randomly replaces *rate* words in the prompt with their synonyms. Synonyms are defined by semantic meaning using the WordNet database [20]. This method is also implemented using NLPaug.

Paraphrasing uses the translation capabilities of LLMs (i.e., Gemini) to paraphrase the prompt. The process aims to generate phrases that maintain the semantic similarity of the prompt while varying their vocabulary, thereby achieving text diversity. The resulting paraphrases are used to evaluate the sensitivity of the LLMs when generating code for diverse prompts that are semantically similar. Figure 1 plots the BERT Score [36] (i.e., semantic similarity) versus the Sacre BLEU [25] (i.e., text diversity) metric for the generated paraphrases for the prompts in Our Dataset. The generated paraphrases are semantically similar and diverse, with BERT score varying between 0.95 and 1.0 and Sacre BLEU metric varying between 0 and 1.0. A lower Sacre BLEU metric value indicates higher diversity.

3.4 Output distance measures

We use the Tree Similarity of Edit Distance (TSED) metric to compute the distance between two LLM code outputs. TSED is a recently published improvement over syntax tree edit distance with an openly available implementation [30]. TSED works as a function $t(c_1, c_2) \rightarrow d \in [0, 1]$, where c_1 and c_2 are fragments of code and d is a real value between 0 and 1 indicating similarity between c_1 and c_2 , 0 meaning completely different and 1 meaning identical. Importantly, we do not evaluate correctness of generated code, and specifically focus on deviations between the codes generated from the original and altered prompts. The motivation behind this decision is that even if the code produced by a model is incorrect, it still represents a possible output the user might see, and therefore is a valid data point when measuring LLM sensitivity.

By its definition TSED measures syntactic differences, not semantic equivalence or functional correctness. Structurally different

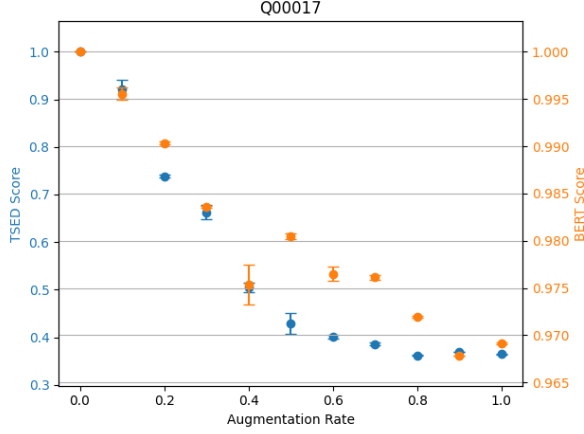


Figure 2: Comparison of BERT and TSED metrics on an example task. While overall trend is comparable, TSED uses most of its possible range, while BERT only varies between 0.96 and 1.0.

code may produce identical outputs, while structurally similar code may behave differently. In this work our focus is on measuring LLM output consistency rather than correctness. Even on its own structural variability matters: users providing similar prompts may receive dramatically different implementations, affecting code review, maintenance, and trust, even when the code is functionally equivalent. Future work should complement structural analysis with functional testing to distinguish benign diversity from problematic instability.

It is also worth noting that we are not using BLEU (quantifying n-grams overlap), BERT score (cosine similarity in BERT contextual embeddings space) or their derivatives to evaluate code similarity. We found these general-purpose text distance metrics to give poor signal when it comes to evaluating generated code. We found BERT to be much better than BLEU, but the results were still poorer than code-specific distance metrics like TSED. Specifically, the mean BERT score was 98.0%, with all 26532 recorded evaluations scoring above 91.0%, suggesting potential range restriction and ceiling effects that could negatively impact our experiments were they to use this metric [23]. An example output comparing BERT and TSED is presented in Figure 2. Another consideration were computational costs, where we found BERT to take 20 times longer to compute, even with access to an NVIDIA A100 GPU. Our observations confirm similar conclusions in earlier studies [8, 30].

4 Experiments

We have evaluated 4 LLMs using the pipeline described above:

- GPT-4o mini
- Claude 3 Haiku
- Gemini 2.0 Flash
- Llama 3.3 70B

To reduce stochasticity of the results, we repeated each request to a LLM 5 times and recorded all responses, and we have also set the

temperature of all models to 0. To ensure all requests produce valid code, we added prefixes and postfixes, unaltered and ignored in the analysis, that stressed to the LLM that each request represents a coding task.

It is worth noting that the evaluation pipeline has no dependency on a particular model, and therefore the exact same evaluations can be carried out for any of the modern LLMs.

4.1 Overall evaluation

We begin by presenting overall evaluation results, aggregated over all datasets. These results are shown in Figures 3 and 4.

All models exhibit similar behavior with respect to **keyboard typos** augmentation (Figure 3, left), with code similarity dropping rapidly between 0.0 and 0.6 augmentation rate. After reaching a TSED value of approximately 0.3 the decay plateaus. As high augmentation rates with typos essentially lead to unreadable prompts, it is not surprising to see approximately 0.3 TSED score, indicating cardinally different code (compared to the one generated from original prompts).

The observed behavior differs significantly with **synonyms augmentation** (Figure 3, right), suggesting synonyms being a much weaker prompt augmentation method. After an initial rapid drop at rate 0.1, code outputs vary slightly, eventually dropping just below 0.5 TSED score for Claude 3 Haiku and GPT-4o mini. Gemini 2.0 Flash is the most resilient to this augmentation method, showing higher overall robustness and never dropping below 0.6 in code similarity compared to unaltered prompt’s output.

Paraphrasing experiment shows behaviour similar to synonyms, see Figure 4. We observe significant drop from the original prompts to low augmentation level, followed by a very gradual decay in code similarity. We can conclude that both synonyms and paraphrasing are much weaker ways of corrupting the prompts, and LLMs are much more robust to such fluctuations in the input compared to typos.

Interestingly, Gemini 2.0 Flash and GPT-4o mini exhibit significantly higher stability for prompts with no augmentation in all experiments, showing 0.9 similarity score with no significant uncertainty. This means these models, with temperature set to 0, output nearly identical codes for the same unaltered prompts. Llama 3.3 and Claude 3 Haiku, on the other hand, exhibit higher instability for unaltered prompts, again with temperature set to 0.

We conducted a Friedman statistical test [9] to evaluate differences in code similarity across all augmentation rates for this experiment. The test revealed a statistically significant effect of augmentation rate on code similarity ($p < 0.001$ for **keyboard typos** and **synonyms**, $p < 0.02$ for **paraphrasings**), indicating a strong relationship between augmentation rate and LLM code generation sensitivity.

4.2 Performance between datasets

Results of evaluating sensitivity on different datasets are shown in Figure 5

These results illustrate the data contamination phenomenon, previously mentioned in Section 3.2. Experiments show the lowest sensitivity on **LeetCode (Old)** dataset across the entire range of augmentation. Because problems in this dataset where most likely

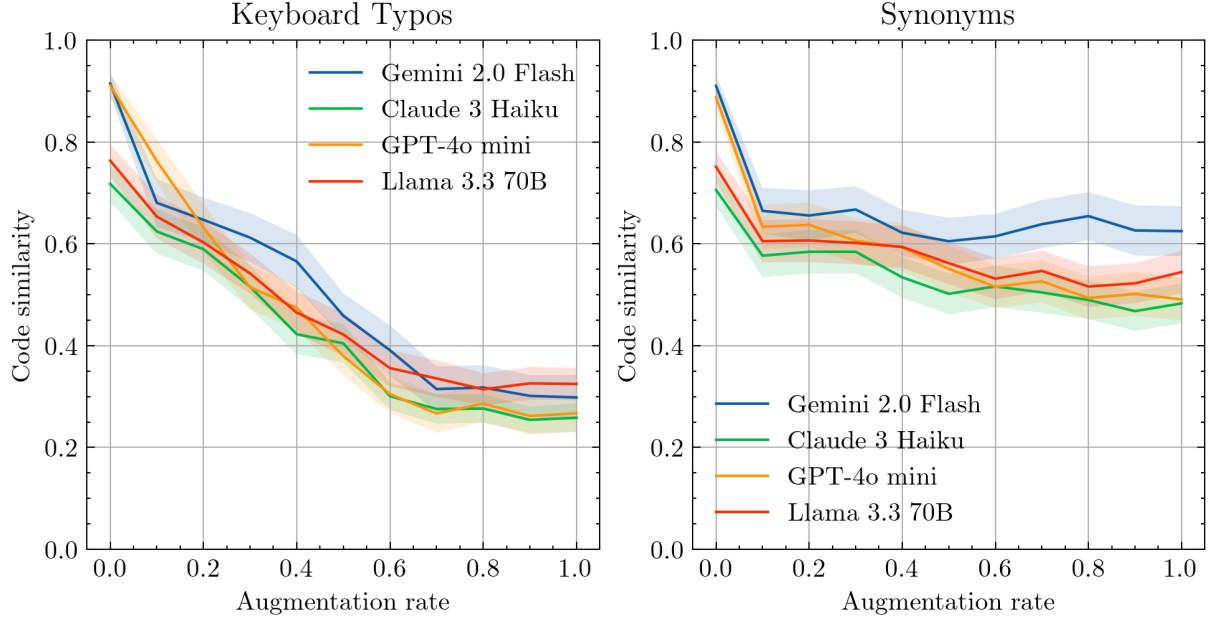


Figure 3: Overall evaluation results. Solid lines represent mean values and shaded regions the 95% intervals, calculated from the set of approximately 3400 observations for each rate step. We can confirm that all models exhibit similar sensitivity to prompt augmentations, with Keyboard Typos being a more invasive augmentation method. Gemini 2.0 Flash is the most robust to synonym augmentation, while sensitivity of all models to typos is approximately the same.

included in training of the evaluated models, LLMs can recognise them from comparatively weak signals, showing exceptional robustness even at higher augmentation rates. This issue is discussed in detail by Sendyka et al. [29].

This behaviour changes when we consider **LeetCode (New)** dataset — code similarity drops lower for both augmentation methods, as these problems were not included in the training data of any of the evaluated models. Nevertheless, newer LeetCode problems still exhibit a lot of traits of standard coding exercises, and therefore all models still exhibit remarkable stability, only dropping below 0.5 similarity score after 50% of the prompt is altered.

The instability is much more severe on **Our Dataset**, where problems do not look like a typical coding exercise from LeetCode or competitive programming, and most likely do not correspond directly to any problem found in LLMs’ training data. Not only do we observe high variance for even unaltered prompts (only 0.7 similarity between generated code outputs), we can also confirm that code similarity drops below 0.5 TSED after only 10% of the prompt is modified, suggesting high sensitivity to these modifications.

Taking into account relatively small sizes of our evaluation datasets (not more than 22 tasks per dataset), we have conducted Kruskal-Wallis H-test [16] to test the difference in sensitivity across datasets. The test showed significant ($p < 0.001$ for **keyboard typos** and $p < 0.02$ for **synonyms**) difference in the distribution of sensitivity increases across three datasets, supporting the generalizability of our pipeline for a variety of programming tasks.

5 Related work

Since LLMs became widely accessible and popular for code generation tasks, the community has focused on their evaluation [4]. This paper extends previous work by proposing an evaluation pipeline for LLM code generation that enables measuring sensitivity to prompt augmentation. This section presents an analysis of the most relevant works in this area.

Initial benchmarks, such as HumanEval [5], SWE-bench [14], and Mostly Basic Programming Problems [1], enable measuring if generated code meets a programming specification. However, these benchmarks do not include variable prompts and are unfit for the rapidly changing research area, because their datasets and pipelines are static. Some works propose tailored pipelines and frameworks to assess LLMs’ code generation capabilities and address the limitations of these initial efforts with more sophisticated and dynamic pipelines. CodeVisionary [33] is an agent-based evaluation framework for complex code generation. CodeVisionary relies on a two-step pipeline that includes requirements context distillation and fine-grained scoring. The first step focuses on requirement comprehension and code generation. The second step involves different agents assessing the generated code and reporting the results. CodeArena [6] is an online evaluation platform that periodically integrates novel coding tasks in an open repository and offers programming interfaces (APIs) for users’ interaction. LiveCodeBench [13] is a holistic evaluation benchmark that continuously updates the coding problems from contests across three coding practice platforms (LeetCode, AtCoder, CodeForces). This

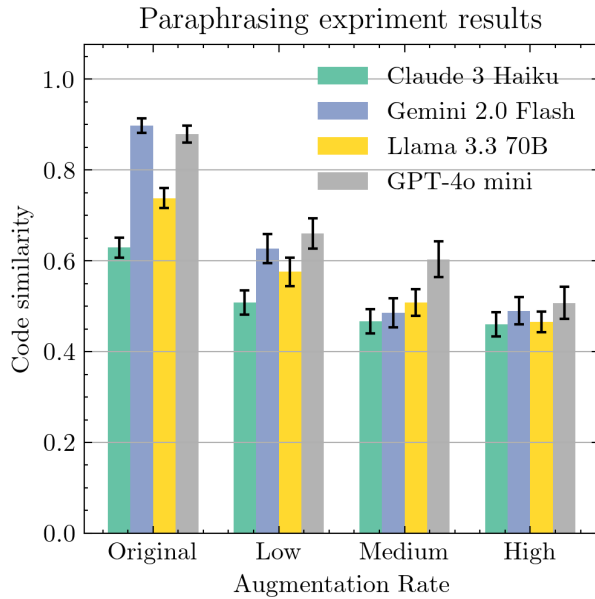


Figure 4: Results of evaluating the LLM models with paraphrasing augmentation. X-axis shows four different levels of paraphrasing: original (unaltered) as well as low (0.5 - 1.0 BLEU distance), medium (0.2 - 0.5), and high (0.0 - 0.2) from the original. We can see paraphrasing augmentation exhibiting similar trend to synonyms - noticeable drop followed by slow gradual decrease in similarity.

benchmark implements an evaluation pipeline for code generation, execution, and test output prediction tasks. EvoCodeBench [17] is a code generation benchmark that dynamically updates code tasks every six months. The benchmark annotates these tasks using a domain taxonomy to enable domain-specific evaluations. The motivation behind continuous updates of coding tasks is to avoid data contamination. This phenomenon threatens evaluations' reproducibility as benchmarking datasets soon become a part of LLMs training data [29]. Our approach complements these works by including several pipeline stages for prompt augmentation.

Some authors have explored the impact of prompt variations on the code generation results. Murr et al. [21] evaluate the performance of LLMs in generating Python programs for coding tasks. They measure how the level of prompt specificity impacts accuracy, time efficiency, and space efficiency of the generated code. The authors propose four levels of specificity, including prompt only, prompt with tests, prompt tests only, and prompt generic tests (i.e., masking test function names). Results show a significant variation in performance across models with different specificity levels. Khojah et al. [15] evaluate the impact of prompt techniques in LLMs' code generation tasks concerning correctness, similarity, and quality. They create a dataset of 7072 prompts to evaluate prompt techniques such as few-shot, persona, chain-of-thought, function signature, and list of packages. The study shows that prompt techniques, such as few-shot or function signature, positively impact

the correctness of the generated code. The persona technique positively impacts the similarity between the generated code and the baseline. Yeo et al. [35] propose a framework to assess the code generation ability of LLMs that integrate with common coding platforms. This framework includes a prompt generation process that creates prompts with different depths. The first level only specifies the problem description, the second level adds constraints, and the third level adds execution examples. Such heterogeneous prompts enable examining their influence on the resulting code. To our knowledge, the three papers above are the closest to our work. These papers report pipelines and results from evaluating the impact of prompt variations in code generation processes. We extend these works by adding augmentation tasks like paraphrasing beyond the proposed specificity levels and prompting techniques, as well as by utilising TSED as a sensitivity measure.

Practitioners have examined the sensitivity of LLMs in domains other than code. The PromptSET benchmark adopts a methodology of slight modifications to prompts, but it evaluates general tasks [26]. Gulati et al. [10] evaluated LLMs on mathematical problems from the Putnam Competition with variable names and constant alterations. They found that performance drops significantly on altered inputs compared to the originals. Their evaluation technique closely resembles our experiments. Hu et al. [12] performed similar experiments on different datasets, including IMDB Wiki-Data, Opendatasoft. Sclar et al. [28] investigated the importance of prompt formatting for reporting LLM performance. Chan et al. [2] use metamorphic testing to validate the stability of code-generating LLMs. This paper evaluates LLMs' sensitivity to prompt perturbations under a completely different setting, namely, their primary use case is a private software house downloading and pretraining a code generation model to create large software projects, meaning their primary focus is on private models. We propose considering a scenario of the general public interacting with widespread and publicly accessible LLM APIs for relatively simple coding tasks. Together, these efforts provide a complementary coverage of LLMs' code generation sensitivity in different use cases.

6 Threats to validity

In this section we discuss main threats to validity of our study.

Internal validity. Large Language Models are normally trained to encourage variability of their output. While temperature is considered to be the main parameter that gauges how random the output of a LLM is, even setting temperature to 0 does not guarantee fully deterministic results. To alleviate this effect we have generated large quantities of raw LLM responses to aggregate over (reaching approximately 3400 datapoints per single augmentation rate step), thus smoothing over occasional anomalous outputs.

As mentioned before, TSED measures structural similarity, not functional equivalence, creating potential construct validity issues. However, this aligns with our goal of measuring LLM sensitivity and output consistency. Structural variability poses real challenges for users who must understand, debug, and maintain different implementations. We recommend complementing TSED with functional testing when correctness is critical.

External validity. The world of LLMs develops at a rapid pace, with new models, or new versions of existing models, being released

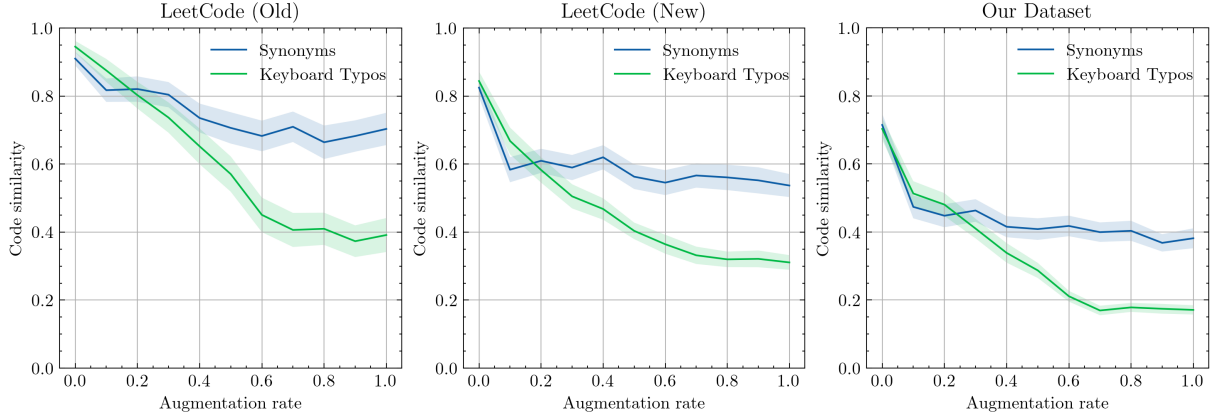


Figure 5: Evaluation results for three datasets used in this study. Solid lines represent mean values and shaded regions the 95% intervals. We can see that LLMs show lowest sensitivity to modifications in tasks from LeetCode (Old) dataset, and highest sensitivity to tasks from dataset we created from scratch.

at frequently⁴. With the constantly moving frontier and relatively slow pace of academic publication, any study on LLMs is bound to be missing most recent releases by the time it is published. Being aware of this risk, we designed our pipeline to be completely agnostic of specific models or API implementations. We have also used multiple LLM providers in all our experiments to show how our method can be applied to different models.

7 Conclusion

Using LLMs to generate code is a proliferating use case. Different users can describe the same code requirement in their prompts in slightly different words, leading to unintentional variation in generated code, thus motivating the need to understand how sensitive LLMs are to fluctuations in prompts. This understanding can help build trust in the code generation process, inform industry providers that create code generation pipelines, and help uncover gaps to motivate further research into LLMs. We have proposed a novel pipeline for evaluating sensitivity of code generation with LLMs to changes in prompts and have shown preliminary evidence that even small fluctuations in prompts such as typos, synonyms, and paraphrasing can impact the generated code.

Concretely, we described an evaluation procedure that uses keyboard typos, synonyms, and paraphrasing to increasingly alter the code task prompt, and uses syntax-tree based code similarity metric to measure changes in the code output. After evaluating specific versions of 4 popular LLMs we find all models to exhibit similar behaviour, showing rapid decay of code similarity for typos augmentation, and being more robust to augmentation with synonyms and paraphrasing. We also illustrate an issue known as data contamination, argue for use of more recent and original tasks to evaluate LLM code generation, and provide a new dataset of open ended tasks.

⁴For example, this Reddit post lists 12 releases in March-May 2024: https://www.reddit.com/r/LocalLLaMA/comments/1cn9sxa/timeline_of_recent_major_llm_releases_past_2/

There can be multiple ways to extend ideas and methods presented in this paper. So far we have only considered one-step interactions with LLMs, while quite often these interactions resemble a dialog. It is therefore reasonable to extend our methods to model such dialogs, using metrics such as the number of information exchanges to measure quality. Similarly, our evaluation can be applied to multi-step pipelines, quantifying uncertainty propagation in such settings. Furthermore, complementing TSED with functional testing would provide a more complete measure of structural and functional variability of the generated code. Finally, it would be valuable to conduct similar experiments involving participants with diverse levels of experience in software engineering, or using augmentation methods that model differences in problem decomposition, mental models, and conceptual framing across users with different backgrounds.

We hope to have provided a deeper insight of the nuanced ways in which prompt variation impacts code generation with LLMs. Ultimately, we aim to inspire future work that enhances the robustness, reliability, and trust in LLM-based code generation systems.

Acknowledgments

This project is supported by funding from the Accelerate Programme for Scientific Discovery, made possible by a donation from Schmidt Sciences.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [2] Pak Yuen Patrick Chan, Jacky Keung, and Zhen Yang. 2025. Effectiveness of symmetric metamorphic relations on validating the stability of code generation LLM. *Journal of Systems and Software* 222 (2025), 112330.
- [3] Junkai Chen, Li Zhenhao, Hu Xing, and Xia Xin. 2025. NLPerturbator: Studying the Robustness of Code LLMs to Natural Language Variations. *ACM Trans. Softw. Eng. Methodol.* (July 2025). doi:10.1145/3745764 Just Accepted.
- [4] Liguang Chen, Qi Guo, Hongrui Jia, Zhengran Zeng, Xin Wang, Yijiang Xu, Jian Wu, Yidong Wang, Qing Gao, Jindong Wang, et al. 2024. A Survey on Evaluating Large Language Models in Code Generation Tasks. *arXiv e-prints* (2024), arXiv–2408.

- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. (2021). arXiv:2107.03374 [cs.LG]
- [6] Mingzhe Du, Anh Tuan Luu, Bin Ji, Xiaobao Wu, Dong Huang, Terry Yue Zhuo, Qian Liu, and See-Kiong Ng. 2025. CodeArena: A Collective Evaluation Platform for LLM Code Generation. *CoRR* abs/2503.01295 (March 2025). <https://doi.org/10.48550/arXiv.2503.01295>
- [7] Ramtin Ehsani, Sakshi Pathak, and Preetha Chatterjee. 2025. Towards detecting prompt knowledge gaps for improved LLM-guided issue resolution. *arXiv [cs.SE]* (Jan. 2025).
- [8] Mikhail Evtikhiev, Egor Bogomolov, Yaroslav Sokolov, and Timofey Bryksin. 2023. Out of the Bleu: how should we assess quality of the code generation models? *Journal of Systems and Software* 203 (2023), 111741.
- [9] Milton Friedman. 1940. A comparison of alternative tests of significance for the problem of m rankings. *The annals of mathematical statistics* 11, 1 (1940), 86–92.
- [10] Aryan Gulati, Brando Miranda, Eric Chen, Emily Xia, Kai Frönsdal, Bruno de Moraes Dumont, and Sanmi Koyejo. 2024. Putnam-AXIOM: A functional and static benchmark for measuring higher level mathematical reasoning. In *The 4th Workshop on Mathematical Reasoning and AI at NeurIPS'24*.
- [11] Ava Heinonen, Bettina Lehtelä, Arto Hellas, and Fabian Fagerholm. 2023. Synthesizing research on programmers' mental models of programs, tasks and concepts – A systematic literature review. *Inf. Softw. Technol.* 164, 107300 (Dec. 2023), 107300.
- [12] Zhibo Hu, Chen Wang, Yanfeng Shu, Hye-Young Paik, and Liming Zhu. 2024. Prompt perturbation in retrieval-augmented generation based large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1119–1130.
- [13] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=chfJJYC3iL>
- [14] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2023. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *The Twelfth International Conference on Learning Representations*.
- [15] Ranim Khojah, Francisco Gomes de Oliveira Neto, Mazen Mohamad, and Philipp Leitner. 2025. The Impact of Prompt Programming on Function-Level Code Generation. *IEEE Transactions on Software Engineering* 51, 8 (2025), 2381–2395. doi:10.1109/TSE.2025.3587794
- [16] William H Kruskal and W Allen Wallis. 1952. Use of ranks in one-criterion variance analysis. *Journal of the American statistical Association* 47, 260 (1952), 583–621.
- [17] Jia Li, Ge Li, Xuanming Zhang, Yunfei Zhao, Yihong Dong, Zhi Jin, Binhua Li, Fei Huang, and Yongbin Li. 2024. EvoCodeBench: An Evolving Code Generation Benchmark with Domain-Specific Evaluations. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 57619–57641.
- [18] Jenny T Liang, Melissa Lin, Nikitha Rao, and Brad A Myers. 2024. Prompts are programs too! Understanding how developers build software containing prompts. *arXiv [cs.SE]* (Sept. 2024).
- [19] Edward Ma. 2019. NLP Augmentation. <https://github.com/makcedward/nlpaug>.
- [20] George A Miller. 1995. WordNet: a lexical database for English. *Commun. ACM* 38, 11 (1995), 39–41.
- [21] Lincoln Murr, Morgan Grainger, and David Gao. 2023. Testing LLMs on code generation with varying levels of prompt specificity. *URL* <https://arxiv.org/abs/2311.07599> (2023).
- [22] Maureen D Neumann, Lisa Dion, and Robert Snapp. 2021. Teaching Computational Thinking. <https://mitpress.mit.edu/9780262045056/teaching-computational-thinking/>. Accessed: 2025-5-15.
- [23] Jum C. Nunnally and Ira H. Bernstein. 1994. *Psychometric Theory* (3rd ed.). McGraw-Hill, New York. xxiv + 752 pages.
- [24] Eric Poitras, Brent Crane, and Angela Siegel. 2024. Generative AI in introductory programming instruction: Examining the assistance dilemma with LLM-based code generators. In *Proceedings of the 2024 on ACM Virtual Global Computing Education Conference V. 1*. ACM, New York, NY, USA, 186–192.
- [25] Matt Post. 2018. A Call for Clarity in Reporting BLEU Scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, Ondřej Bojar, Rajen Chatterjee, Christian Federmann, Mark Fishel, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, Christof Monz, Matteo Negri, Aurélie Nèveol, Mariana Neves, Matt Post, Lucia Specia, Marco Turchi, and Karin Verspoor (Eds.). Association for Computational Linguistics, Brussels, Belgium, 186–191. doi:10.18653/v1/W18-6319
- [26] Amirhossein Razavi, Mina Soltangheis, Negar Arabzadeh, Sara Salamat, Morteza Zihayat, and Ebrahim Bagheri. 2025. Benchmarking prompt sensitivity in large language models. In *European Conference on Information Retrieval*. Springer, 303–313.
- [27] Diana Robinson, Christian Cabrera, Andrew D Gordon, Neil D Lawrence, and Lars Mennen. 2025. Requirements are all you need: The final frontier for end-user software engineering. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–22.
- [28] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2023. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. In *The Twelfth International Conference on Learning Representations*.
- [29] Radzim Sendyka, Christian Cabrera, Andrei Paleyes, Diana Robinson, and Neil Lawrence. 2025. LLM Performance for Code Generation on Noisy Tasks. arXiv:2505.23598 [cs.LG] <https://arxiv.org/abs/2505.23598>
- [30] Yewei Song, Cedric Lothritz, Xunzhu Tang, Tegawendé Bissyandé, and Jacques Klein. 2024. Revisiting Code Similarity Evaluation with Abstract Syntax Tree Edit Distance. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 38–46. doi:10.18653/v1/2024.acl-short.3
- [31] Hari Subramonyam, Divy Thakkar, Andrew Ku, Jürgen Dieber, and Anoop Sinha. 2024. Prototyping with prompts: Emerging approaches and challenges in generative AI design for collaborative software teams. *arXiv [cs.HC]* (Feb. 2024).
- [32] Annapurna Vadaparty, Daniel Zingaro, David H Smith, IV, Mounika Padala, Christine Alvarado, Jamie Gorson Benario, and Leo Porter. 2024. CS1-LLM: Integrating LLMs into CS1 Instruction. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. ACM, New York, NY, USA, 297–303.
- [33] Xincheng Wang, Pengfei Gao, Chao Peng, Ruida Hu, and Cuiyun Gao. 2025. Code-Visionary: An Agent-based Framework for Evaluating Large Language Models in Code Generation. *arXiv e-prints* (2025), arXiv–2504.
- [34] Jeannette M Wing. 2008. Computational thinking and thinking about computing. *Philos. Trans. A Math. Phys. Eng. Sci.* 366, 1881 (Oct. 2008), 3717–3725.
- [35] Sangyeop Yeo, Yu-Seung Ma, Sang Cheol Kim, Hyungkook Jun, and Taehoo Kim. 2024. Framework for evaluating code generation ability of large language models. *ETRI Journal* 46, 1 (2024), 106–117. doi:10.4218/etrij.2023-0357
- [36] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*.
- [37] Ziyin Zhang, Chaoyu Chen, Bingchang Liu, Cong Liao, Zi Gong, Hang Yu, Jianguo Li, and Rui Wang. 2023. Unifying the perspectives of nlp and software engineering: A survey on language models for code. *arXiv preprint arXiv:2311.07989* (2023).